Afk Architecture

AFK 系统架构手册

> AFK 的架构说明与评审材料。 结构：**边界** → **约束** → **质量属性** → **层次** → **闭环** → **不变量** → **决策** → **失败模型** → **演进**。 全部结论经源码核实，不含文件引用。

一句话：AFK 是一个把工单自动化流转为可合并变更的自治执行系统。 它的全部设计压力来自一个前提——**执行者（编码 Agent）不可信**。

---

1. 系统边界

1.1 系统定位

AFK 位于**工单系统**与**代码仓库**之间，对接两侧并托管中间的执行生命周期。

AFK **夹在工单系统与代码仓库之间**，向下对接两侧，向上托管中间的完整执行生命周期：

相邻系统	提供给 AFK	AFK 回写给它
工单系统	工单身份、权威状态、评论通道	状态推进、业务标签、进展记录
代码仓库	分支、提交、变更请求语义	推送分支、创建变更请求
编码执行者	执行能力、原生会话、上报信号	目标、约束、实时控制指令

三条边界关系，各自的信任级别不同：

关系	信任程度	由此得出的设计
与工单系统	**对方权威，且随时可能被人工改动**	本地状态只推动、不覆盖
与代码仓库	语义可靠	借用其版本控制模型
与编码执行者	**不可信**	判定权必须与之分离

1.2 所有权划分

归 AFK 所有	AFK 只借用
执行生命周期与状态推进	工单系统的持久状态（**外部权威**）
执行权独占（租约）	Agent 的执行能力与原生会话
工作区隔离与清理	代码仓库的版本控制语义
执行过程的完整审计	平台 API 的读写通道
验收判定与放行决策	终端复用来实现实时控制
预算与失败计数	——

> **关键点**：AFK **拥有生命周期但不拥有状态。** 工单状态是外部权威，AFK 只是推动它——因为第三方随时可能手动改工单。

1.3 明确的非目标

非目标	理由
不拆分工单、不做需求分解	分解是规划问题，与执行正交
不做版本发布与部署	止于”可合并”，之后是交付流程
不做多租户安全隔离	假设单租户可信环境
不替代人工代码审查	提供可信证据，不替代人的判断
不做实时流式监控	呈现是投影拉取，不是推送

---

2. 约束分析

架构不是从需求推导的，是从**约束**推导的。这部分决定了后面所有决策。

2.1 硬约束

#	约束	推论
C1	**执行者不可信**	完成必须由独立方验证；声明必须强校验
C2	**工单可被人类与外部系统编辑**	外部状态必须权威；本地不得反向覆盖
C3	**执行环境可能崩溃**	跨进程状态必须可持久化、可重放
C4	**上下文上限不可自察**	执行器必须是上下文溢出的唯一权威
C5	**平台未提供原生条件更新**	并发控制需自建，且必须承认单机边界

2.2 由约束推出的结论

这三条是全系统的**派生结论**，也是整套架构最核心的推论：

> **D-1**：执行者不可信（C1）→ **判定权与执行权必须分离**，同一主体不能既实现又验收。
>
> **D-2**：工单可被改（C2）→ **本地投影永不反向写入外部权威**，只能推动。
>
> **D-3**：环境会崩（C3）→ **过程必须先于结果可追溯**，否则事后无法解释。

---

3. 质量属性与取舍优先级

> **这是架构文档的核心。** 架构的本质是"在冲突的品质之间做有意识的排序"。

排名	品质	含义	为何排这个位置
1	**保守失败**	不确定时停下，而非继续	错误的自动化副作用远比人工介入昂贵
2	**判定可信**	"完成"必须可验证	整个系统的产出价值系于此
3	**可恢复**	崩溃后可解释、可续跑	执行者是 AI，不崩溃才是异常
4	**可审计**	任何执行可事后解释	自动化系统不 explain 就不敢放开
5	**可移植**	跨工单源、Agent、沙箱	平台会变，Agent 会变
6	**一致性及时性**	最终一致即可	主动排在最后

3.1 明确的取舍

得到	牺牲	典型体现
判定正确	**吞吐**	就绪判定逐项拉取依赖，用调用次数换正确性
判定正确	**实时性**	呈现层拉取投影而非订阅事件流
安全性	**自动化程度**	租约不确定 → 阻塞转人工，而非自动重试
可解释	**简洁**	自建文件租约而非"直接上分布式锁"
数据完整性	**可演进性**	哈希链下不引入迁移，旧事件永久保留
执行可靠性	**资源成本**	质检串行、交接留快照

> **结论**："我们的性能不是最好的，但它的每一个性能损失都是一次明确的、可解释的决策。 如果我给你看一张性能优化清单，那意味着我还没想清楚哪些东西不能省。"

3.2 不追求的品质

品质	决策
最小化架构	**拒绝**。复杂度是本质复杂度，不是实现冗余
零依赖	**部分追求**。领域核心与图编译层保持零依赖，但不为此接受重复实现
实时性	**明确放弃**。最终一致足够
高可用	**单机假设**。多机靠平台原生能力，不自建

---

4. 层次结构

4.1 六层栈

序	层	承载	一句话职责
⑥	**交互层**	呈现	命令行、实时看板、桌面工位；只读投影 + 人工接管入口
⑤	**技能层**	**知识**	任务分解、实现、质检、产出；编码"怎么做"
④	**契约层**	语义	完成协议、验收标准、审计事件、能力契约
③	**控制平面**	策略	工单接入、调度抢占、生命周期、工作流编排
②	**工具基座**	能力	进程与终端控制、工作区、沙箱与凭据、遥测
①	**环境**	事实	宿主系统、容器运行时、代码仓库、工单平台接口

层与层之间只有两条通道：**向下派发指令，向上接收举证**。没有跨层直连。

4.2 每层编码什么

> 这是分层的**根本区分**：不同层回答的是不同性质的问题。

层	回答的问题	性质
⑥ 交互层	怎么看	**呈现**
⑤ 技能层	**怎么做**	**知识**（可变）
④ 契约层	什么算完成 / 什么算证据	**语义**
③ 控制平面	能不能做 / 何时必须停	**策略**
② 工具基座	能不能执行 / 能否打断	**能力**
① 环境	物理约束	**事实**

> **一句话**：技能层是**知识**，控制平面是**策略**，工具基座是**能力**，契约层是**语义**。混淆这四者，就会写出"把业务规则塞进 Agent 提示词"这类设计。

4.3 变更频率矩阵（分层的真正目的）

层	谁可以改	变更频率	可否被绕过
⑥ 交互	前端 / 产品	中	**不可绕过**（无决策权可绕）
⑤ 技能	**业务 / 用户随时**	**高**	可替换，**骨架不可绕过**
④ 契约	架构评审	**低**	**不可绕过**
③ 控制	架构评审	低	不可绕过
② 工具	平台适配	中	不可绕过
① 环境	不可控	—	—

> **这是整个架构最重要的一句话：技能层是全栈中唯一被设计为"高频可替换"的一层。**因为**工艺（怎么做）天天改进，而骨架（能不能做、结果算不算数）不能天天改。**分层的全部意义就是——**把高频变更隔离在可替换层，把必须守住的规则固化在不可绕过层。**

4.4 技能层的架构定位

这是 AFK 最有辨识度的设计，必须讲透：

	技能层	系统层（③+④+②）
承载什么	**HOW** —— 工艺、步骤、话术、约束提示	**WHETHER / WHEN** —— 能不能做、何时停
可信度	**不可信**，由人编写、可被用户替换	**不可绕过**，代码强制
失效后果	做不好，但**不会破坏系统**	破坏系统不变量
变更节奏	**持续演进**	谨慎变更，需评审

推论：系统层不假设技能层会遵守规则。**即使技能写错了**——漏步骤、写错命令、乱改文件——租约不会失效、审计不会停、预算不会重置、协议不会放行。

> **这是"用不可信层承载高频变更、用强制层兜底"的标准做法。**类比：技能层像**应用代码**，系统层像**运行时**。你不会因为应用代码写得烂就放弃运行时保护。

技能层的两条特殊护栏

护栏	作用
能力围栏	以工具禁用清单与"禁止自动调用"开关在框架层约束，**不靠提示词**
知识保留规则	通用抽象可泛化；**项目专属契约必须原样保留**

> 知识保留规则针对的是 **LLM 特有的失败模式**：模型会把项目契约当成通用知识"优化"掉。 讲这个能直接体现你真的踩过坑。

4.5 工具基座（Harness）的抽象边界

工具基座**不理解业务**。它只提供六件事：

能力	说明
拉起	启动一个执行者进程，注入目标与约束
供给	准备隔离工位、沙箱与凭据
监听	采集上报信号（完成声明、用量、存活）
控制	实时投递输入、中断、终止
观察	捕获过程产出，供事后解释
清理	释放资源，按策略决定保留或移除工位

> **关键区分**：**中断 ≠ 终止**。 中断是优雅停止，预留恢复可能；终止是强制杀死，不假设可恢复。 这个区分决定了上下文交接能不能实现。

4.6 端口与正交扩展轴

控制平面与工具基座之间是**端口与适配器**：编排只依赖接口，不依赖实现。

端口	已实现适配器
工单源 / 分支源 / 变更源	GitLab、GitHub、Git
执行者源	**六种 Agent CLI**
沙箱源	**本地、容器两种引擎**
租约	文件系统租约（平台 CAS 为预留端口）
事件存储 / 证据存储	只追加日志存储 / 本地加密存储

四条正交扩展轴，任意组合构成一条产线：

轴	消除哪一类差异	已实现规模
执行者源	各家 Agent 的参数、输出格式、能力差异	六种
沙箱源	运行环境的隔离级别与控制方式	三种
分支策略	分支命名、基线选取、并行隔离	四种
工作流模板	业务步骤、依赖、条件分支	多套内置

> **为什么必须正交**：曾把"执行者差异""环境差异""业务流程差异"揉进一个类型， 导致任一维度变化都要改类定义，组合也无法穷举验证。

能力契约是插件接入方式：能力集合是**封闭集**， 同一能力被两个插件提供 → **启动失败**，不做静默覆盖或优先级仲裁。

> **为什么冲突要失败**：能力冲突意味着**运行时语义不确定**。 一个以"可预测"为前提的系统，不能在启动期就引入语义歧义。

---

5. 闭环的构造

> **有层次不等于有系统。闭环才让它成为系统。**

5.1 上下行信号的不对称

这是整个闭环最核心的纪律：

方向	内容	性质	校验强度
下行（授权）	工单目标、阶段指令、约束条件	授权	授权范围内即可
上行（举证）	完成声明、用量上报、存活信号、审计事件	**举证**	**必须严格强校验**

> **举证必须比授权严格。** 下行发错，最多浪费一次执行；上行放过一次错误声明，**后面所有环节都建立在谎言上。**

5.2 主回路

主回路分六步，**每一步都跨越一个层**，这也是闭环得以成立的机制：

步	发生在	动作	跨层性质
①	控制平面	工单就绪判定	——
②	控制平面	租约抢占 + 就绪校验（四条件）	策略层决定**能否开始**
③	工具基座	隔离工位、沙箱、凭据注入	能力层兑现**执行条件**
④	技能层	工艺执行：编排 → 动作 → 代码变更	知识层决定**怎么做**
⑤	契约层	完成声明的强校验	语义层决定**算不算数**
⑥	契约层	独立质检重新验收	语义层 + 策略层决定**放不放行**

分支：通过则推送变更进入待合并，人工合并后经关联对账确认进入完成； 不通过则写入返工记录，开启**一次新的尝试**回到第 ① 步。

贯穿全程：每一次状态推进都向审计上下文投递一条事件。 这些事件既是本次执行的**输出证明**，也是下一轮的**输入依据**——返工记录决定下一轮重做什么， 预算快照决定还能不能继续转。**闭环由此闭合。**

5.3 六条回路

闭环不止一条——**不同失败粒度回到不同的位置**：

回路	触发	回到哪	换的是什么	语义
执行重试	进程崩溃 / 协议非法	③ 工艺执行	**换会话**	同一次尝试
上下文	用量越阈值	③ 工艺执行	**换代次**	同一次尝试
返工	质检失败	① 工单就绪	**换尝试**	同一张工单
人工	阻塞转人工	① 工单就绪	—	人解除阻塞后重入
预算	预算耗尽	阻塞	—	**元回路**：决定其他回路还能转几次
审计	全程	事后 + 下次输入	—	让回路可被追溯

> **最易被追问的点**：**重试、上下文、返工三者的粒度不同**—— 前两者换的是**执行细节**（会话 / 代次），返工换的是**执行整体**（新的一次尝试）。 混为一谈会导致返工次数被高估、或上下文交接被当成失败重来。

预算回路是元回路：它不推动流程，而是决定其他回路还能转几次。 这解释了为什么预算必须持久化——**重启若清零，元回路就被绕过了。**

5.4 闭环的闭合条件

什么保证它真的闭，而不是一条会断的流水线：

#	条件	否则
1	每次状态推进都写审计事件	循环不可追溯，事故无法解释
2	失败不丢现场	转人工后无人能接续
3	计数跨重启持久化	**重启即可绕过上限**
4	权威单一	循环里出现两个真相

> **闭环的闭合不是靠"跑完了"，而是靠"任何一步失败都知道该回哪去、且回得去"。** 审计事件流既是环的**输出证明**，也是环的**下次输入**（返工记录、预算快照）—— **这才是真正的闭环，而非流水线。**

5.5 一次完整执行的信号流

时点	方向	信号	谁产生	谁消费	强校验?
派发	下行	目标与约束	控制平面	技能层	授权内即可
执行中	上行	用量上报	执行者	控制平面	**是**（越阈值即中断）
执行中	上行	存活心跳	工具基座	租约	**是**（丢失即封锁）
完成	上行	完成声明	执行者	契约层	**是**（不合法即失败）
验收	—	验收结论	独立验证者	控制平面	**是**
全程	上行	审计事件	全层	审计上下文	**是**（失败即阻断）

> **没有一个上行信号是"信任即可"的。** 这是闭环可信的全部来源。

---

6. 核心不变量

> **不变量是架构的真正内容**——组件可换、代码可重写，不变量不能破。

#	不变量	违反后果
I1	外部工单状态是生命周期**唯一权威**，本地不得反向写入	人工操作被覆盖，信任崩塌
I2	一次执行尝试**至多一个持有者**	双写同一工单，产生冲突变更
I3	租约归属**不确定时不得抢占**	活跃执行者被取代，双写
I4	完成必须经**独立验证者**确认	自评偏差，未完成被当作完成
I5	完成声明必须通过**协议强校验**；协议不合法 ≠ 通过	格式错误的"完成"被误判成功
I6	审计事件**不可修改、不可删除**	失去可审计性
I7	事件持久化失败**必须阻断后续外部副作用**	审计无法证明"未记录的事未发生"
I8	缺失数据建模为**"未知"**，不退化为"否"或"零"	陈旧数据被当作"已确认正常"
I9	预算**跨执行累加**，重启不重置	重启即绕过上限，无限消耗
I10	呈现层**不具备决策权**	界面行为影响业务状态
I11	**技能层可替换，但不可绕过骨架**	工艺改写顺带改掉了系统不变量

> **I7 最值得展开**：若审计允许失败后继续， 这份审计**永远无法证明"没有记录的事发生过"**——这与"日志能证明什么"是同一条线。

6.1 生命周期状态机

状态集：就绪 · 进行中 · 验证中 · 待合并 · 完成 · 阻塞 · 返工

合法迁移（其余一律拒绝）：

#	从	到	触发	守卫条件
1	就绪	进行中	抢占成功	四条件全满足
2	返工	进行中	抢占成功	四条件全满足
3	进行中	验证中	实现阶段完成	验收通过
4	验证中	待合并	质检通过	返工次数未超限
5	验证中	返工	质检失败	返工次数未超限
6	返工	阻塞	返工次数达上限	——
7	待合并	完成	关联对账确认	人工已合并且关联正确
8	进行中 / 验证中	阻塞	失败 · 超时 · 冲突 · **租约不确定**	——

终态是完成与阻塞。终态之后拒绝任何迁移——这条守卫不可绕过。

可运行的四个合取条件（缺一不可，依赖读取失败按不可执行处理）：

状态为「就绪」或「返工」 ｜ 执行模式为「自动」 ｜ **无未完成子项**（父项永不可执行） ｜ **全部依赖已完成**

进入「阻塞 + 转人工」的四条路径，铁律是**绝不静默重试为自动工作**：

失败 ｜ 超时 ｜ 冲突 ｜ **租约不确定**

> 最后一条最关键：租约过期有两种成因——持有者已死（该回收） 与持有者仍存活但卡死或失联（**回收即双写**）。分不清就不该回收。

---

7. 架构决策记录

> 每条按 **背景** → **决策** → **否决的备选** → **后果** 四段陈述。 “否决了什么”往往比”选了什么”更能说明判断力。

D-01　判定权与执行权分离

背景：执行者不可信（C1），系统需要一个”完成”信号。 **决策**：实现与验收是两个**独立主体**，验证者重新执行验收。 **否决**：实现者自评（偏差无法消除）；覆盖率代替验收（覆盖率高≠语义正确）。 **后果**：单次执行成本翻倍。**这是刻意付出的代价——**判定可信排在吞吐之前。

D-02　外部状态权威，本地仅推动

背景：工单可被人类与外部系统编辑（C2）。 **决策**：工单状态是唯一权威（I1），本地状态只影响展示健康度。 **否决**：以本地为准回写覆盖（会覆盖人工操作）；要求强一致（人工编辑不可禁止）。 **后果**：会出现”外部已完成、本地仍显示进行中”。**可接受——**投影可陈旧，权威不能错。

D-03　技能层与技术层正交，护栏在框架层

背景：技能是 prompt，天然不可信。 **决策**：技能层承载 HOW 且高频可替换；系统层不可绕过。 **护栏**：工具禁用清单、禁止自动调用、**知识保留规则。** **否决**：仅靠提示词约束（模型不保证遵守）；仅靠全局禁止（无法差异化）。 **后果**：新增技能需理解围栏语义。收益是安全策略**可审计且真正生效。**

D-04　不引入分布式锁，接受单机边界

背景：平台未提供原生条件更新（C5）。 **决策**：自建文件系统租约串行化抢占临界区，并在文档中**显式声明其为单机机制。** **否决**：分布式锁服务——单机场景引入新的可用性依赖，锁服务挂了执行停摆， 而文件锁的失效模式是”退化成抢不到”，**更安全**；进程内互斥——无法覆盖多进程。 **后果**：**多机执行不受支持**，必须走平台原生能力。已知边界，但必须写进文档避免误判。

D-05　原子发布完整记录，而非原子创建空标记

背景：锁的内容本身携带所有权信息。 **决策**：先在私有位置准备好**完整**记录，再通过一次**原子重命名**发布到锁路径。 **否决**：独占创建标志文件——只能原子产生空文件，内容需二次写入， **存在半初始化窗口**；先写再判断——检查与写入间存在竞态。 **后果**：锁路径上出现的一定是完整状态。配套要求：清理前**重新比对身份**。

D-06　过期租约 fail-closed

背景：租约过期的原因不可从本地状态区分。 **决策**：过期租约**不允许直接抢占**（I3），必须先由权威侧持久化阻塞才批准清理。 **否决**：过期即可抢占（“已死”与”失联”不可区分，抢占即双写）；超时自动重试（违反铁律）。 **后果**：宿主异常退出后工单需人工恢复。**一次人工确认远低于一次双写事故**。

D-07　预算持久化，且账目损坏 fail-closed

背景：重启是最常见的失败模式；上下文消耗直接对应成本。 **决策**：预算按工单持久化，按会话**只接受增长、按差值累加**（I9），校验「明细求和 = 总量」，不成立即拒绝执行。 **否决**：内存态（等于给无限消耗开后门）；损坏重置为满额（同义反复）；清零重来（等于满额）。 **后果**：账目损坏会阻塞该工单。 **对比**：会话快照损坏则 **fail-open**，降级为文档交接。 **同一架构里两种相反策略**——这正是”每个持久化边界都要显式决定失败方向”。

D-08　审计不可变，且写入失败阻断业务

背景：外部副作用必须可解释（I6、I7）。 **决策**：事件只追加、带哈希链、不可改删；持久化是后续外部副作用的**前置条件**。 **否决**：尽力而为——若审计可失败后继续，它就**永远无法证明”没有记录的事发生过”**； 先业务后补审计（无法保证顺序完整性）。 **后果**：事件流限定为诊断、对账与比对，**不作为自动合并或状态恢复的唯一依据**。 **关联取舍**：哈希链保完整性，代价是**无迁移层**，旧事件永久保留。

D-09　呈现层只读，投影拉取替代事件订阅

背景：多前端与执行进程需共享状态。 **决策**：前端只读（I10），数据来自本地投影**拉取**，不建立订阅通道。 **否决**：前端订阅事件（引入中间件与顺序一致性负担，诊断场景收益有限）； 前端直读权威源（等于给界面越权）。 **后果**：最终一致。**这是把一致性及时性排在最后一位的直接结果**（§3）。

D-10　桌面端复用同一批用例，渲染层零平台能力

背景：桌面端需要工单与执行历史。 **决策**：**直接调用同一批用例**，不派生执行进程；渲染层零系统能力， 所有访问经主进程通道白名单与类型锁定接口。 **否决**：调用命令行取 *JSON*（增加进程边界与序列化开销，两条路径可能漂移）；渲染层直读文件（越权）。 **后果**：桌面端须自带依赖打包，构建链更重。收益是**两条客户端走同一代码路径。**

---

8. 失败模型

> 成熟的系统不是”不会失败”，而是**知道自己会怎么失败**。

失败类型	检测方式	系统响应	终态
执行者进程崩溃	进程退出且无有效声明	执行级重试，换会话	上限后阻塞
完成声明格式非法	协议解析 fail-closed	**判失败**，不视为通过	触发重试
验收不通过	独立质检返回失败	返工记录 + 重跑实现	超限阻塞
上下文耗尽	执行器轮询越阈值	中断 → 交接 → 新代次	预算耗尽则阻塞
超时（单代次 / 整体）	看门狗 / 累计预算	阻塞转人工，**保留现场**	阻塞
代码冲突	合并冲突检测	阻塞转人工，保留现场	阻塞
租约丢失	心跳失败 / 租约过期	**封锁，禁止抢占**	阻塞
重复抢占	抢占原子失败	干净退化为”抢不到”	无副作用
审计写入失败	写入异常	**阻断后续全部外部副作用**	失败并清理
凭据或平台不可用	接口错误	退避重试，**不推进状态**	保持原状态
投影数据损坏	校验失败	降级为已验证前缀 + 诊断	局部不可用
预算账目损坏	一致性校验失败	**拒绝执行**	阻塞
会话快照损坏	校验失败	**降级文档交接**	继续执行

8.1 失败响应的三条原则

从这张表能读出三条一致的原则——**这比逐条记忆更有价值**：

原则	体现
不确定时停止，而非继续	租约丢失、账目损坏、审计失败 → 全部阻塞
代价越大的失败，保留越多现场	超时 / 冲突保留工位，便于人工接续
可降级的不阻断，不可降级的必阻断	会话快照降级（fail-open）；审计账本阻断（fail-closed）

> **结论**：“失败处理不是逐个 *case* 决定的，是三条原则推出来的。 你可以问我任何一个失败场景，我都能告诉你它落在哪条原则上。”

---

9. 安全边界

> “AI 在你机器上执行任意代码”是所有安全质询的起点，必须主动讲。

边界	定位
本地模式是任务隔离，非安全隔离	能防执行者互相干扰，**不能**阻止其访问宿主其他文件、进程、环境变量
容器模式才具备安全边界	仅挂载工作区与会话、结果三个路径
凭据走显式白名单	白名单**可扩展、不可收缩**；移除默认项须 fork 配置
单租户假设	不处理同机多互不信任租户

> **设计取向**：让收紧成为**显式且困难**的动作。 核心是**最小权限**而非”多写检查”。

要点：“我们把’本地模式的隔离级别’写进了设计文档的显式声明， 而不是让人默认以为跑在容器里。安全设计的底线是——**不能让使用者误判。**”

---

10. 演进路径与已知边界

10.1 已经发生的判断反转

曾经的判断	现在的判断	触发原因
保留命令别名以兼容旧用法	**彻底移除**，无兼容层	别名让已删命令仍活在文档里，破坏单一真相源
保留全局调度器扫描全部就绪工单	**只接受单工单 + 显式授权范围**	全局扫描无法回答"这次执行对应哪个授权范围"
分离的实现队列与质检队列	**统一执行路径**	两套队列产生两套状态真相
执行者通过提示词主动写控制信号	**改为执行器侧事件流 + 结构化结果**	提示词不是控制通道
保留历史遗留层	**遗留清单清零**	遗留基线非空就只增不减，最终腐化
保留"审计模式"开关	**无条件强制审计**	默认安全 > 可测试性
事件流支持版本迁移	**不迁移，保完整性**	哈希链与迁移的延续规则难以自洽

> **最后两条的范式值得讲**：“把'默认不安全'换成'强制安全'，再用依赖注入解决由此产生的可测试性问题，而不是留一个开关把问题推迟。”

10.2 已知边界与未兑现项

> **原则：主动暴露 > 被动承认。** 每条按 **已知边界 / 明确的下一步 / 我的取舍** 三类定性——不含辩解。

#	边界	定性	表述
1	平台原生条件更新未落地	已知边界	“接口与策略分支就绪、测试注入假实现，真实平台条件更新未接。**多机执行目前不支持，这是已知边界不是缺陷。**”
2	文件租约非多机共识	已知边界	“只保证单机 + 本地可信文件系统。威胁模型写在注释与架构文档中。”
3	**验收标准的检查命令未被系统执行**	**明确的下一步**	“系统侧只做协议层强校验。真正的机器校验需进沙箱执行命令并回收证据——**这是质量上限的主要瓶颈**。”
4	能力配置的 schema 校验未接线	明确的下一步	“为保持 SDK 零依赖手写 schema 子集，实际校验未接线；示例配置无加载层。”
5	脱敏非通用 DLP	已知边界	“规则覆盖主流凭据格式，人名、内部地址处理不了。文档已声明。”
6	事件 schema 无迁移	**我的取舍**	“哈希链与迁移有张力。**保完整性 > 可演进性**，代价是旧流永久保留。”
7	交互式接管尚未最终验收	已知边界	“调度不变性仍是未关闭的验收项。”
8	部分决策记录文本损坏	我的失误	“写入时的工具问题。本次结论均由实现反推，未引用损坏段落。”
9	文档漂移	我的失误	“两处：诊断子命令文案与实现不一致；技能规范字段列表与实际使用不符。链接检查只能抓断链，抓不到陈旧措辞。”
10	存在重复实现	我的取舍	“字段规范化写了两份，因为零依赖包抽

#	边界	定性	表述
			不到公共层。**这是零依赖换来的重复。**”
11	就绪判定存在 N+1 调用	我的取舍	“用调用次数换正确性。”
12	投影缓存以修改时间判失效	我的取舍	“同秒内两次写入可能返回陈旧结果。”

10.3 下一个真正的瓶颈

> 架构师应当能指出**限制系统继续前进的是什么**，而不是罗列待办事项。

判定可信度是当前上限。 系统能安全执行、能审计、能恢复， 但”验收标准是否真的被机器执行”尚未闭环——**这决定了自动化产出能不能被真正信任。** 相比之下，性能、并发度、界面体验都是次要问题。

其次是**多机执行**：租约的单机性质决定了横向扩展必须等平台原生能力， 这不是优化工作，是**架构前提的变更。**

---

11. 常见问题与论证要点

A. 定位与分层

Q1. 用一句话讲这个系统。 > 一个把工单自动化流转为可合并变更的自治执行系统。 > 全部设计压力来自一个前提——**执行者不可信**。

Q2. 分几层？每层回答什么问题？ > 见 §4.2。关键区分：**技能层是知识，控制平面是策略，工具基座是能力，契约层是语义。** > 混淆这四者就会写出"把业务规则塞进提示词"这类设计。

Q3. 技能层在架构里是什么位置？ > **全栈中唯一被设计为高频可替换的一层**（§4.3）。 > 因为工艺天天改进，骨架不能天天改。 > 技能层承载 HOW，系统层承载 WHETHER/WHEN； > **技能层不可信但可替换，系统层不可绕过**（I11）。

Q4. 凭什么说技能层不可信系统能兜住？ > 因为租约不会因技能写错而失效、审计不会停、预算不会重置、协议不会放行。 > 护栏在框架层而非提示词层——工具禁用清单、禁止自动调用、知识保留规则（D-03）。

Q5. 系统边界在哪？什么归你管？ > AFK **拥有生命周期但不拥有状态。**工单状态是外部权威（人随时会改）， > 本地运行时状态只影响展示健康度（D-02）。

Q6. 为什么不用现成方案？ > 现成方案解决"让 Agent 写代码"；本系统解决"让不可信的 Agent 在受控系统里产出可信结果"。 > **后者是分布式系统的题，不是 AI 的题。** 判定权分离、租约防重入、 > 审计追溯、进程退出语义拆分——这四项现成方案都不提供。

B. 闭环

Q7. 这个系统怎么闭环？ > 见 §5.2 主回路 + §5.3 六条回路。 > 关键是**不同失败粒度回到不同位置**：重试换会话、上下文换代次、返工换尝试。

Q8. 为什么上下行信号校验强度不同？ > **举证必须比授权严格。** 下行发错最多浪费一次执行； > 上行放过一次错误声明，后面所有环节都建立在谎言上（§5.1、§5.5）。

Q9. 什么保证它真的闭？ > 四个条件：每次推进写审计、失败不丢现场、计数跨重启持久化、权威单一（§5.4）。 > **闭合不是靠"跑完了"，是靠"任何一步失败都知道该回哪去、且回得去"。**

Q10. 怎么处理失败？ > 不是逐个 case 决定，是三条原则推出：**不确定时停止、代价越大的失败保留越多现场、 > 可降级的不阻断而不可降级必阻断**（§8.1）。

C. 架构决策

Q11. 最重要的一个决策是什么？ > 判定权与执行权分离（D-01）。约束 C1 是”执行者不可信”， > 同一主体既实现又验收偏差无法消除。代价是成本翻倍，我把判定可信排在吞吐前面。

Q12. 为什么不用分布式锁？ 见 D-04 **Q13. 租约的原子性怎么保证？** 见 D-05 **Q14. 租约过期直接抢行不行？** 见 D-06 **Q15. 预算为什么必须持久化？** 见 D-07 **Q16. 为什么审计写失败要阻断业务？** > 若审计允许失败后继续，它就永远无法证明”没有记录的事发生过”（D-08）。 **Q17. 架构怎么保证不腐化？** > 守卫脚本在 CI 中拒绝反向依赖、禁止源码中的泛型逃逸口； > 历史遗留清单已**清零**。要点：**“架构约定写在文档里没用，我把它变成了会失败的检查。”**

D. 不变量

Q18. 核心不变量是什么？ 见 §6 十一条。**优先讲 I1、I7、I9、I11。 Q19. 为什么缺失数据要建模成”未知”而不是”否”？** > “否”是一个**断言**，缺失只是**没有观测到**。混同会让陈旧数据被当作”已确认正常”（I8）。

E. Agent 工程

Q20. Agent 不靠谱怎么办？“完成”怎么定义？ > 三重校验：① 结构化声明而非自然语言；② 解析 fail-closed，**协议不合法 ≠ 通过**； > ③ 独立验证者重新执行。**最终判定 = 执行者声明 × 协议强校验**。

Q21. 上下文满了怎么办？Agent 自己知道吗？ > 不知道——这正是约束 C4，所以执行器是唯一权威。超阈值 → 中断 → 交接 → 新代次续跑。

Q22. 三个重试上限的区别？ > 执行级（换会话）／自纠错（**保留现场**）／质检返工（**计数持久化跨重启**）。 > 差别在于是否保留现场、计数是否跨执行累加。

Q23. 接入新 Agent 成本大吗？ > 中等：命令构造与输出解析需单独实现，但**能力矩阵必须显式声明**。 > 不支持续跑或结构化输出的 Agent 不得错误进入续跑流程——**禁止乐观假设**。

F. 质量属性与工程

Q24. 这个系统牺牲了什么？ > §3.1：**吞吐换正确、实时性换一致性、自动性换安全性、简洁性换可解释、完整性换可演进性**。 > 结论：“我们的性能不是最好的，但每个性能损失都是一次明确的决策。”

Q25. 测试策略？ > 四层：领域纯函数 → 平台契约 → 真终端集成 → 假执行者端到端。 > **架构守卫本身也有测试**——保证”不许腐化”这件事本身被测到。

Q26. 本地模式安全吗？ > **任务隔离，不是安全隔离**（§9）。必须主动讲。

Q27. 你改过什么判断？ 见 §10.1 的七条反转。 **Q28. 如果重做，哪里会改？** > 优先补齐验收标准的真实机器执行——**这是当前判定可信度的上限**（§10.3）。

---

12. 核心陈述要点

12.1 六条浓缩结论

> 1. **“进程退出不等于工作完成”** —— D-02 的通俗版
> 2. **“日志不是权威，事件流才是”** —— I6 的推论
> 3. **“缺失即未知，而不是否”** —— I8
> 4. **“执行者是上下文溢出的唯一权威”** —— C4
> 5. **“技能层可替换，骨架不可绕过”** —— I11
> 6. **“举证必须比授权严格”** —— §5.1

12.2 如何说明系统的可信度

不要只报测试数量，要说清**测试的层次**—— 领域纯函数 → 平台契约 → 真终端集成 → 假执行者端到端， 以及”**架构守卫本身也有测试**”。

12.3 应当避免的表述

- ❌ “这个项目很简单，就是调 API” —— 规模与复杂度都不小
- ❌ “Agent 嘛，不可靠是正常的，能跑就行” —— **不可靠恰恰是设计的第一前提**
- ❌ 编造没做过的优化或指标 —— 这套系统的文档诚实度很高，吹出来反而露馅

12.4 深入阅读的优先级

优先精读两块：**租约实现**与**审计事件存储**。 这两处能解释绝大多数深度追问的答案。